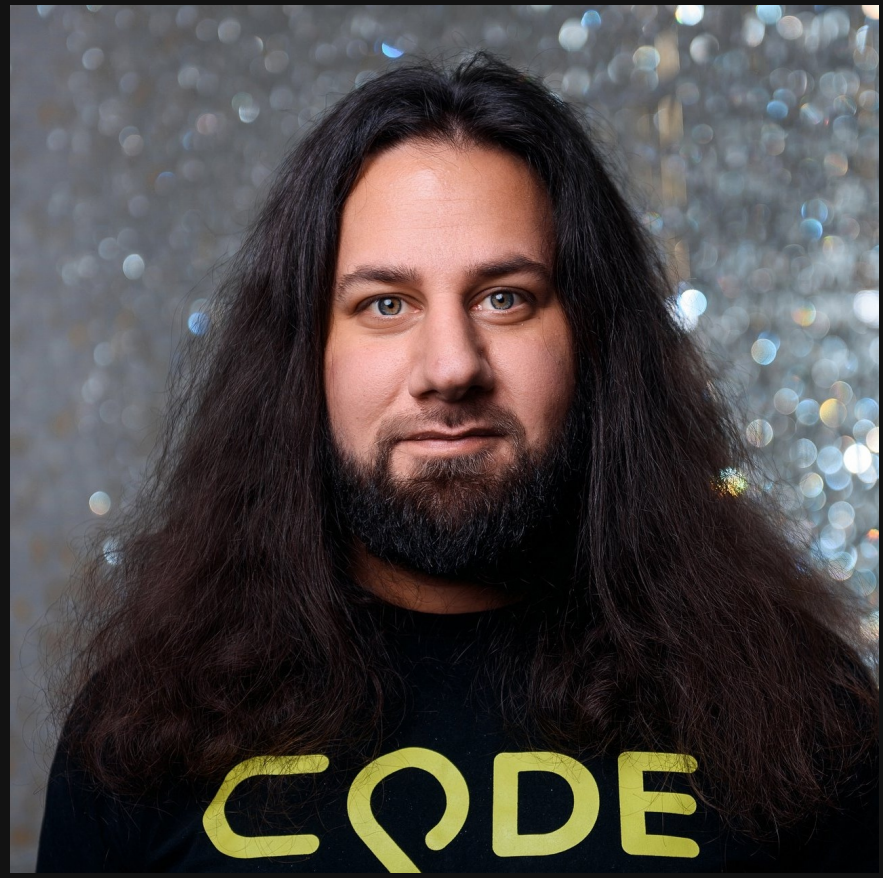
Kotlin Goodies for Testing

Pasha Finkelshteyn, 

`whoami`

- Kotlin Java CSS Vue developer
- Average Python enjoyer
- Developer avocado at JetBrains
- Data engineer
- Speaker and streamer Twitch (https://twitch.tv/jetbrains or Twitch.tv/JetBrains)



All programming activities have one common trait

They require testing!

Who are you?

1. You write code/tests in Java or Kotlin
2. You wanna migrate to Kotlin, *or*
3. You wanna make your tests in Kotlin better, *or*
4. You wanna introduce Kotlin in your projects!

How are we doing it?

1. Using DSLs (domain specific languages)
2. Structuring our code better
3. Using smart libraries

BDD

1. Given
2. When
3. Then



Cucumber

```
Feature: An example
  Scenario: The example
    Given I have a username "pasha"
    When I type username in field "x"
    And Press OK button
    Then I get an error "Enter password"
```

What’s the problem here?

Cucumber

```
1  Feature: An example
2    Scenario: The example
3      Given I have a username "pasha"
4      When I type username in field "x"
5      And Press OK button
6      Then I get an error "Enter password"
```

This line might intersect with **any** test

Cucumber

```
1  Feature: An example
2    Scenario: The example
3      Given I have a username "pasha"
4      When I type username in field "x"
5      And Press OK button
6      Then I get an error "Enter password"
```

Actually **any** line can

Cucumber

```
Feature: An example
  Scenario: The example
    Given I have a username "pasha"
    When I type username in field "x"
    And Press OK button
    Then I get an error "Enter password"
```

Actually **any** line can

Steps match by **name** O_o:

```
@And("Press OK button")
void press_ok_button(){/*TODO*/}
```

How can we do it in Kotlin?

Kotest: https://kotest.io

- No obscure text files
- No string matching
- everything in one place

```
1   class MyTests : BehaviorSpec({
2       given("a broomstick") {
3           `when`("I sit on it") {
4               then("I should be able to fly") {
5                   // test code
6               }
7           }
8           `when`("I throw it away") {
9               then("it should come back") {
10                  // test code
11              }
12          }
13      }
14  })
```

Kotest

Behavior Spec

Popular with people who like to write tests in the *BDD* style, `BehaviorSpec` allows you to use `given`, `when`, `then`.

```
class MyTests : BehaviorSpec({
    given("a broomstick") {
        `when`("I sit on it") {
            then("I should be able to fly") {
                // test code
            }
        }
        `when`("I throw it away") {
            then("it should come back") {
                // test code
            }
        }
    }
})
```

How can we do it in Kotlin?

Kotest: https://kotest.io

- No obscure text files
- No string matching
- everything in one place

```
class MyTests : BehaviorSpec({
    given("a broomstick") {
        `when`("I sit on it") {
            then("I should be able to fly") {
                // test code
            }
        }
        `when`("I throw it away") {
            then("it should come back") {
                // test code
            }
        }
    }
})
```

Kotest

Behavior Spec

Popular with people who like to write tests in the *BDD* style, `BehaviorSpec` allows you to use `given`, `when`, `then`.

```
class MyTests : BehaviorSpec({
    given("a broomstick") {
        `when`("I sit on it") {
            then("I should be able to fly") {
                // test code
            }
        }
        `when`("I throw it away") {
            then("it should come back") {
                // test code
            }
        }
    }
})
```

How can we do it in Kotlin?

Kotest: https://kotest.io

- No obscure text files
- No string matching
- everything in one place

```
class MyTests : BehaviorSpec({
    given("a broomstick") {
        `when`("I sit on it") {
            then("I should be able to fly") {
                // test code
            }
        }
        `when`("I throw it away") {
            then("it should come back") {
                // test code
            }
        }
    }
})
```

Kotest

Behavior Spec

Popular with people who like to write tests in the *BDD* style, `BehaviorSpec` allows you to use `given`, `when`, `then`.

```
class MyTests : BehaviorSpec({
    given("a broomstick") {
        `when`("I sit on it") {
            then("I should be able to fly") {
                // test code
            }
        }
        `when`("I throw it away") {
            then("it should come back") {
                // test code
            }
        }
    }
})
```

How can we do it in Kotlin?

Kotest: https://kotest.io

- No obscure text files
- No string matching
- everything in one place

```
class MyTests : BehaviorSpec({
    given("a broomstick") {
        `when`("I sit on it") {
            then("I should be able to fly") {
                // test code
            }
        }
        `when`("I throw it away") {
            then("it should come back") {
                // test code
            }
        }
    }
})
```

Kotest

Behavior Spec

Popular with people who like to write tests in the *BDD* style, `BehaviorSpec` allows you to use `given`, `when`, `then`.

```
class MyTests : BehaviorSpec({
    given("a broomstick") {
        `when`("I sit on it") {
            then("I should be able to fly") {
                // test code
            }
        }
        `when`("I throw it away") {
            then("it should come back") {
                // test code
            }
        }
    }
})
```

How can we do it in Kotlin?

Kotest: https://kotest.io

- No obscure text files
- No string matching
- everything in one place

```
class MyTests : BehaviorSpec({
    given("a broomstick") {
        `when`("I sit on it") {
            then("I should be able to fly") {
                // test code
            }
        }
        `when`("I throw it away") {
            then("it should come back") {
                // test code
            }
        }
    }
})
```

Kotest

Behavior Spec

Popular with people who like to write tests in the *BDD* style, `BehaviorSpec` allows you to use `given`, `when`, `then`.

```
class MyTests : BehaviorSpec({
    given("a broomstick") {
        `when`("I sit on it") {
            then("I should be able to fly") {
                // test code
            }
        }
        `when`("I throw it away") {
            then("it should come back") {
                // test code
            }
        }
    }
})
```

How can we do it in Kotlin?

Kotest: https://kotest.io

- No obscure text files
- No string matching
- everything in one place

```
class MyTests : BehaviorSpec({
    given("a broomstick") {
        `when`("I sit on it") {
            then("I should be able to fly") {
                // test code
            }
        }
        `when`("I throw it away") {
            then("it should come back") {
                // test code
            }
        }
    }
})
```

Kotest

Behavior Spec

Popular with people who like to write tests in the *BDD* style, `BehaviorSpec` allows you to use `given`, `when`, `then`.

```
class MyTests : BehaviorSpec({
    given("a broomstick") {
        `when`("I sit on it") {
            then("I should be able to fly") {
                // test code
            }
        }
        `when`("I throw it away") {
            then("it should come back") {
                // test code
            }
        }
    }
})
```

How can we do it in Kotlin?

Kotest: https://kotest.io

- No obscure text files
- No string matching
- everything in one place

```
1   class MyTests : BehaviorSpec({
2       given("a broomstick") {
3           `when`("I sit on it") {
4               then("I should be able to fly") {
5                   // test code
6               }
7           }
8           `when`("I throw it away") {
9               then("it should come back") {
10                  // test code
11              }
12          }
13      }
14  })
```

Kotest

Behavior Spec

Popular with people who like to write tests in the *BDD* style, `BehaviorSpec` allows you to use `given`, `when`, `then`.

```
class MyTests : BehaviorSpec({
    given("a broomstick") {
        `when`("I sit on it") {
            then("I should be able to fly") {
                // test code
            }
        }
        `when`("I throw it away") {
            then("it should come back") {
                // test code
            }
        }
    }
})
```

Another flavor of Kotest

Kotest: https://kotest.io

```
class MyTests : DescribeSpec({
    describe("score") {
        it("start as zero") { /* test here */ }
        describe("with a strike") {
            it("adds ten") { /* test here */ }
            it("carries strike to the next frame") { /* test here */ }
        }
        describe("for the opposite team") {
            it("Should negate one score") { /* test here */ }
        }
    }
})
```

Another flavor of Kotest

Kotest: https://kotest.io

```
class MyTests : DescribeSpec({
    describe("score") {
        it("start as zero") { /* test here */ }
        describe("with a strike") {
            it("adds ten") { /* test here */ }
            it("carries strike to the next frame") { /* test here */ }
        }
        describe("for the opposite team") {
            it("Should negate one score") { /* test here */ }
        }
    }
})
```

Another flavor of Kotest

Kotest: https://kotest.io

```
class MyTests : DescribeSpec({
    describe("score") {
        it("start as zero") { /* test here */ }
        describe("with a strike") {
            it("adds ten") { /* test here */ }
            it("carries strike to the next frame") { /* test here */ }
        }
        describe("for the opposite team") {
            it("Should negate one score") { /* test here */ }
        }
    }
})
```

Another flavor of Kotest

Kotest: https://kotest.io

```
class MyTests : DescribeSpec({
    describe("score") {
        it("start as zero") { /* test here */ }
        describe("with a strike") {
            it("adds ten") { /* test here */ }
            it("carries strike to the next frame") { /* test here */ }
        }
        describe("for the opposite team") {
            it("Should negate one score") { /* test here */ }
        }
    }
})
```

Another flavor of Kotest

Kotest: https://kotest.io

```
class MyTests : DescribeSpec({
    describe("score") {
        it("start as zero") { /* test here */ }
        describe("with a strike") {
            it("adds ten") { /* test here */ }
            it("carries strike to the next frame") { /* test here */ }
        }
        describe("for the opposite team") {
            it("Should negate one score") { /* test here */ }
        }
    }
})
```

Migrating from JUnit?

```
class AnnotationSpecExample : AnnotationSpec() {
    @BeforeEach
    fun beforeTest() {
        println("Before each test")
    }
    @Test
    fun test1() {
        1 shouldBe 1
    }
    @Test
    fun test2() {
        3 shouldBe 3
    }
}
```

Migrating from JUnit?

```
class AnnotationSpecExample : AnnotationSpec() {
    @BeforeEach
    fun beforeTest() {
        println("Before each test")
    }
    @Test
    fun test1() {
        1 shouldBe 1
    }
    @Test
    fun test2() {
        3 shouldBe 3
    }
}
```

Migrating from JUnit?

```
class AnnotationSpecExample : AnnotationSpec() {
    @BeforeEach
    fun beforeTest() {
        println("Before each test")
    }
    @Test
    fun test1() {
        1 shouldBe 1
    }
    @Test
    fun test2() {
        3 shouldBe 3
    }
}
```

Migrating from JUnit?

```
class AnnotationSpecExample : AnnotationSpec() {
    @BeforeEach
    fun beforeTest() {
        println("Before each test")
    }
    @Test
    fun test1() {
        1 shouldBe 1
    }
    @Test
    fun test2() {
        3 shouldBe 3
    }
}
```

Migrating from JUnit?

```
class AnnotationSpecExample : AnnotationSpec() {
    @BeforeEach
    fun beforeTest() {
        println("Before each test")
    }
    @Test
    fun test1() {
        1 shouldBe 1
    }
    @Test
    fun test2() {
        3 shouldBe 3
    }
}
```

Migrating from JUnit?

```
class AnnotationSpecExample : AnnotationSpec() {
    @BeforeEach
    fun beforeTest() {
        println("Before each test")
    }
    @Test
    fun test1() {
        1 shouldBe 1
    }
    @Test
    fun test2() {
        3 shouldBe 3
    }
}
```

50 shades of kotest

https://kotest.io

Fun spec	String Spec
Should spec	Describe spec
Behaviour spec	Word spec
Free spec	Feature spec
Expect spec	Annotation spec

How do I run the same tests with different inputs?

Data-driven testing a.k.a Parametrized tests

Parametrized tests? Java

```
static Stream<Arguments> stringIntAndListProvider() {
    return Stream.of(
        arguments("apple", 1, Arrays.asList("a", "b")),
        arguments("lemon", 2, Arrays.asList("x", "y"))
    );
}
@ParameterizedTest
@MethodSource("stringIntAndListProvider")
void testWithMultiArgMethodSource(String str, int num, List<String> list) {/
    * snip */
}
```

Parametrized tests? Java

```
static Stream<Arguments> stringIntAndListProvider() {
    return Stream.of(
        arguments("apple", 1, Arrays.asList("a", "b")),
        arguments("lemon", 2, Arrays.asList("x", "y"))
    );
}
@ParameterizedTest
@MethodSource("stringIntAndListProvider")
void testWithMultiArgMethodSource(String str, int num, List<String> list) {/
    * snip */
}
```

Parametrized tests? Java

```java
static Stream<Arguments> stringIntAndListProvider() {
    return Stream.of(
        arguments("apple", 1, Arrays.asList("a", "b")),
        arguments("lemon", 2, Arrays.asList("x", "y"))
    );
}
@ParameterizedTest
@MethodSource("stringIntAndListProvider")
void testWithMultiArgMethodSource(String str, int num, List<String> list) {/
    * snip */
}
```

Parametrized tests? Java

```java
static Stream<Arguments> stringIntAndListProvider() {
    return Stream.of(
        arguments("apple", 1, Arrays.asList("a", "b")),
        arguments("lemon", 2, Arrays.asList("x", "y"))
    );
}
@ParameterizedTest
@MethodSource("stringIntAndListProvider")
void testWithMultiArgMethodSource(String str, int num, List<String> list) {/
    * snip */
}
```

Parametrized tests? Java

```java
static Stream<Arguments> stringIntAndListProvider() {
    return Stream.of(
        arguments("apple", 1, Arrays.asList("a", "b")),
        arguments("lemon", 2, Arrays.asList("x", "y"))
    );
}
@ParameterizedTest
@MethodSource("stringIntAndListProvider")
void testWithMultiArgMethodSource(String str, int num, List<String> list) {/
    * snip */
}
```

Parametrized tests? Java

```java
static Stream<Arguments> stringIntAndListProvider() {
    return Stream.of(
        arguments("apple", 1, Arrays.asList("a", "b")),
        arguments("lemon", 2, Arrays.asList("x", "y"))
    );
}
@ParameterizedTest
@MethodSource("stringIntAndListProvider")
void testWithMultiArgMethodSource(String str, int num, List<String> list) {/
    * snip */
}
```

Parametrized tests? Java

```java
static Stream<Arguments> stringIntAndListProvider() {
    return Stream.of(
        arguments("apple", 1, Arrays.asList("a", "b")),
        arguments("lemon", 2, Arrays.asList("x", "y"))
    );
}
@ParameterizedTest
@MethodSource("stringIntAndListProvider")
void testWithMultiArgMethodSource(String str, int num, List<String> list) {/
    * snip */
}
```

Parametrized tests? Java

```java
static Stream<Arguments> stringIntAndListProvider() {
    return Stream.of(
        arguments("apple", 1, Arrays.asList("a", "b")),
        arguments("lemon", 2, Arrays.asList("x", "y"))
    );
}
@ParameterizedTest
@MethodSource("stringIntAndListProvider")
void testWithMultiArgMethodSource(String str, int num, List<String> list) {/
    * snip */
}
```

Parametrized tests? Java

```
static Stream<Arguments> stringIntAndListProvider() {
    return Stream.of(
        arguments("apple", 1, Arrays.asList("a", "b")),
        arguments("lemon", 2, Arrays.asList("x", "y"))
    );
}
@ParameterizedTest
@MethodSource("stringIntAndListProvider")
void testWithMultiArgMethodSource(String str, int num, List<String> list) {/
    * snip */
}
```

Parametrized tests? Java

```java
static Stream<Arguments> stringIntAndListProvider() {
    return Stream.of(
        arguments("apple", 1, Arrays.asList("a", "b")),
        arguments("lemon", 2, Arrays.asList("x", "y"))
    );
}
@ParameterizedTest
@MethodSource("stringIntAndListProvider")
void testWithMultiArgMethodSource(String str, int num, List<String> list) {/
    * snip */
}
```

`stringIntAndListProvider` ,really?

Parametrized tests? Java

```java
static Stream<Arguments> stringIntAndListProvider() {
    return Stream.of(
        arguments("apple", 1, Arrays.asList("a", "b")),
        arguments("lemon", 2, Arrays.asList("x", "y"))
    );
}
@ParameterizedTest
@MethodSource("stringIntAndListProvider")
void testWithMultiArgMethodSource(String str, int num, List<String> list) {/
    * snip */
}
```

`stringIntAndListProvider` ,really?

- Becomes cluttered

Parametrized tests? Java

```java
static Stream<Arguments> stringIntAndListProvider() {
    return Stream.of(
        arguments("apple", 1, Arrays.asList("a", "b")),
        arguments("lemon", 2, Arrays.asList("x", "y"))
    );
}
@ParameterizedTest
@MethodSource("stringIntAndListProvider")
void testWithMultiArgMethodSource(String str, int num, List<String> list) {/
    * snip */
}
```

`stringIntAndListProvider`, really?

- Becomes cluttered
- Not typed (enough)

Parametrized tests? Java

```java
static Stream<Arguments> stringIntAndListProvider() {
    return Stream.of(
        arguments("apple", 1, Arrays.asList("a", "b")),
        arguments("lemon", 2, Arrays.asList("x", "y"))
    );
}
@ParameterizedTest
@MethodSource("stringIntAndListProvider")
void testWithMultiArgMethodSource(String str, int num, List<String> list) {/
    * snip */
}
```

`stringIntAndListProvider`, really?

- Becomes cluttered
- Not typed (enough)
- Bound by a string!

Parametrized tests? Kotlin (with kotest)

```
fun isPythagTriple(a: Int, b: Int, c: Int): Boolean = a * a + b * b == c * c
// snip…
withData( // ddt
  Triple(3, 4, 5),
  Triple(6, 8, 10),
  Triple(8, 15, 17),
  Triple(7, 24, 25)
) { (a, b, c) -> // destructuring
  isPythagTriple(a, b, c) shouldBe true //test
}
```

Parametrized tests? Kotlin (with kotest)

```kotlin
fun isPythagTriple(a: Int, b: Int, c: Int): Boolean = a * a + b * b == c * c
// snip…
withData( // ddt
  Triple(3, 4, 5),
  Triple(6, 8, 10),
  Triple(8, 15, 17),
  Triple(7, 24, 25)
) { (a, b, c) -> // destructuring
  isPythagTriple(a, b, c) shouldBe true //test
}
```

Parametrized tests? Kotlin (with kotest)

```kotlin
fun isPythagTriple(a: Int, b: Int, c: Int): Boolean = a * a + b * b == c * c
// snip…
withData( // ddt
  Triple(3, 4, 5),
  Triple(6, 8, 10),
  Triple(8, 15, 17),
  Triple(7, 24, 25)
) { (a, b, c) -> // destructuring
  isPythagTriple(a, b, c) shouldBe true //test
}
```

Parametrized tests? Kotlin (with kotest)

```kotlin
fun isPythagTriple(a: Int, b: Int, c: Int): Boolean = a * a + b * b == c * c
// snip…
withData( // ddt
  Triple(3, 4, 5),
  Triple(6, 8, 10),
  Triple(8, 15, 17),
  Triple(7, 24, 25)
) { (a, b, c) -> // destructuring
  isPythagTriple(a, b, c) shouldBe true //test
}
```

Parametrized tests? Kotlin (with kotest)

```kotlin
fun isPythagTriple(a: Int, b: Int, c: Int): Boolean = a * a + b * b == c * c
// snip…
withData( // ddt
  Triple(3, 4, 5),
  Triple(6, 8, 10),
  Triple(8, 15, 17),
  Triple(7, 24, 25)
) { (a, b, c) -> // destructuring
  isPythagTriple(a, b, c) shouldBe true //test
}
```

Parametrized tests? Kotlin (with kotest)

```kotlin
fun isPythagTriple(a: Int, b: Int, c: Int): Boolean = a * a + b * b == c * c
// snip…
withData( // ddt
  Triple(3, 4, 5),
  Triple(6, 8, 10),
  Triple(8, 15, 17),
  Triple(7, 24, 25)
) { (a, b, c) -> // destructuring
  isPythagTriple(a, b, c) shouldBe true //test
}
```


What if
I need to test things I don’t know I need to test?

Never write your own hashing (unless…)

```
withData(
    "love",
    "secret",
    "god", // Who will remember the reference?
) { input ->
    MD5.compute(input) shouldBe libMd5(input)
}
```

Never write your own hashing (unless…)

```
withData(
    "love",
    "secret",
    "god", // Who will remember the reference?
) { input ->
    MD5.compute(input) shouldBe libMd5(input)
}
```

Never write your own hashing (unless…)

```
withData(
    "love",
    "secret",
    "god", // Who will remember the reference?
) { input ->
    MD5.compute(input) shouldBe libMd5(input)
}
```

Never write your own hashing (unless…)

```
withData(
    "love",
    "secret",
    "god", // Who will remember the reference?
) { input ->
    MD5.compute(input) shouldBe libMd5(input)
}
```

Never write your own hashing (unless…)

```
withData(
    "love",
    "secret",
    "god", // Who will remember the reference?
) { input ->
    MD5.compute(input) shouldBe libMd5(input)
}
```

Launching…

Never write your own hashing (unless…)

```
withData(
    "love",
    "secret",
    "god", // Who will remember the reference?
) { input ->
    MD5.compute(input) shouldBe libMd5(input)
}
```

Launching…

1. ✔ love
2. ✔ secret
3. ✔ god


Time to investigate!

Property tests!

To test things we can not even *imagine*

```
checkAll(
    Arb.string(
        codepoints = Arb.of(
            (Char.MIN_VALUE.code..Char.MAX_VALUE.code).map(::Codepoint)
        ),
        range = 0..52
    )
) { input ->
    MD5.compute(input) shouldBe libMd5(input)
}
```

Property tests!

To test things we can not even *imagine*

```
checkAll(
    Arb.string(
        codepoints = Arb.of(
            (Char.MIN_VALUE.code..Char.MAX_VALUE.code).map(::Codepoint)
        ),
        range = 0..52
    )
) { input ->
    MD5.compute(input) shouldBe libMd5(input)
}
```

Property tests!

To test things we can not even *imagine*

```
checkAll(
    Arb.string(
        codepoints = Arb.of(
            (Char.MIN_VALUE.code..Char.MAX_VALUE.code).map(::Codepoint)
        ),
        range = 0..52
    )
) { input ->
    MD5.compute(input) shouldBe libMd5(input)
}
```

Property tests!

To test things we can not even *imagine*

```
checkAll(
    Arb.string(
        codepoints = Arb.of(
            (Char.MIN_VALUE.code..Char.MAX_VALUE.code).map(::Codepoint)
        ),
        range = 0..52
    )
) { input ->
    MD5.compute(input) shouldBe libMd5(input)
}
```

Property tests!

To test things we can not even *imagine*

```
checkAll(
    Arb.string(
        codepoints = Arb.of(
            (Char.MIN_VALUE.code..Char.MAX_VALUE.code).map(::Codepoint)
        ),
        range = 0..52
    )
) { input ->
    MD5.compute(input) shouldBe libMd5(input)
}
```

Property tests!

To test things we can not even *imagine*

```
checkAll(
    Arb.string(
        codepoints = Arb.of(
            (Char.MIN_VALUE.code..Char.MAX_VALUE.code).map(::Codepoint)
        ),
        range = 0..52
    )
) { input ->
    MD5.compute(input) shouldBe libMd5(input)
}
```

Property tests!

To test things we can not even *imagine*

```
checkAll(
    Arb.string(
        codepoints = Arb.of(
            (Char.MIN_VALUE.code..Char.MAX_VALUE.code).map(::Codepoint)
        ),
        range = 0..52
    )
) { input ->
    MD5.compute(input) shouldBe libMd5(input)
}
```

Property tests!

To test things we can not even *imagine*

```
checkAll(
    Arb.string(
        codepoints = Arb.of(
            (Char.MIN_VALUE.code..Char.MAX_VALUE.code).map(::Codepoint)
        ),
        range = 0..52
    )
) { input ->
    MD5.compute(input) shouldBe libMd5(input)
}
```



Error!

```
Property test failed for inputs

0) "▫Ⱡ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫Ấ▫▫▫▫▫Ở▫▫▫▫▫▫▫▫▫ḙ▫▫▫▫"

Caused by io.kotest.assertions.AssertionFailedError: expected:<"90b23a970ed1b799850a24140ae8ad39"> but was:<"5a430f76
```

Wow, it was a complex sample!

But there is more...

Error!

```
Property test failed for inputs

0) "▫Ⱡ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫Ấ▫▫▫▫▫Ở▫▫▫▫▫▫▫▫▫ỵ▫▫▫▫"

Caused by io.kotest.assertions.AssertionFailedError: expected:<"90b23a970ed1b799850a24140ae8ad39"> but was:<"5a430f76
```

Wow, it was a complex sample!

But there is more...

Error!

```
Property test failed for inputs

0) "▫Л▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫Ấ▫▫▫▫▫Ở▫▫▫▫▫▫▫▫▫ñ▩▩▩▩"

Caused by io.kotest.assertions.AssertionFailedError: expected:<"90b23a970ed1b799850a24140ae8ad39"> but was:<"5a430f76
```

Wow, it was a complex sample!

But there is more...

Error!

```
Property test failed for inputs

0) "▫Ⱡ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫Ấ▫▫▫▫▫Ỏ▫▫▫▫▫▫▫▫▫ῆ▩▩▩▩"

Caused by io.kotest.assertions.AssertionFailedError: expected:<"90b23a970ed1b799850a24140ae8ad39"> but was:<"5a430f76
```

Wow, it was a complex sample!

But there is more...

Error!

```
Attempting to shrink arg "▫Ѧ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫Ấ▫▫▫▫▫Ở▫▫▫▫▫▫▫▫▫ḧ▩▩▩▩"
Shrink #1: "▫Ѧ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▩▩▩" fail
Shrink #2: "▫Ѧ▫▫▫▫▫▫▫▫▩✱" fail
Shrink #3: "▫Ѧ▫▫▫▩" fail
Shrink #4: "▫Ѧ▩" fail
Shrink #5: "▫Ѧ" fail
Shrink #6: "▩" fail
Shrink #7: <empty string> pass
Shrink result (after 7 shrinks) => "▩"
```

And there is more!

Error!

```
Attempting to shrink arg "▫Ѧ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫Ấ▫▫▫▫▫Ở▫▫▫▫▫▫▫▫▫ḧ▩▩▩▩"
Shrink #1: "▫Ѧ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▩▩▩" fail
Shrink #2: "▫Ѧ▫▫▫▫▫▫▫▫▩*" fail
Shrink #3: "▫Ѧ▫▫▫▩" fail
Shrink #4: "▫Ѧ▩" fail
Shrink #5: "▫Ѧ" fail
Shrink #6: "▩" fail
Shrink #7: <empty string> pass
Shrink result (after 7 shrinks) => "▩"
```

And there is more!

Error!

```
Attempting to shrink arg "▫Ʌ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫Ấ▫▫▫▫▫Ở▫▫▫▫▫▫▫▫▫�somethingXXXX"
Shrink #1: "▫Ʌ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫XXX" fail
Shrink #2: "▫Ʌ▫▫▫▫▫▫▫▫X*" fail
Shrink #3: "▫Ʌ▫▫▫X" fail
Shrink #4: "▫ɅX" fail
Shrink #5: "▫Ʌ" fail
Shrink #6: "X" fail
Shrink #7: <empty string> pass
Shrink result (after 7 shrinks) => "X"
```

And there is more!

Error!

```
Attempting to shrink arg "▫Ʌ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫Ấ▫▫▫▫▫Ở▫▫▫▫▫▫▫▫▫�somethingXXXX"
Shrink #1: "▫Ʌ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫XXX" fail
Shrink #2: "▫Ʌ▫▫▫▫▫▫▫▫X*" fail
Shrink #3: "▫Ʌ▫▫▫X" fail
Shrink #4: "▫ɅX" fail
Shrink #5: "▫Ʌ" fail
Shrink #6: "X" fail
Shrink #7: <empty string> pass
Shrink result (after 7 shrinks) => "X"
```

And there is more!

Error!

```
Attempting to shrink arg "▫Λ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫Ấ▫▫▫▫▫Ở▫▫▫▫▫▫▫▫▫ḧ▩▩▩▩"
Shrink #1: "▫Λ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▩▩▩" fail
Shrink #2: "▫Λ▫▫▫▫▫▫▫▫▩*" fail
Shrink #3: "▫Λ▫▫▫▩" fail
Shrink #4: "▫Λ▩" fail
Shrink #5: "▫Λ" fail
Shrink #6: "▩" fail
Shrink #7: <empty string> pass
Shrink result (after 7 shrinks) => "▩"
```

And there is more!

Error!

```
Attempting to shrink arg "▫Ѧ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫Ấ▫▫▫▫▫Ở▫▫▫▫▫▫▫▫▫ḧ▩▩▩▩"
Shrink #1: "▫Ѧ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▩▩▩" fail
Shrink #2: "▫Ѧ▫▫▫▫▫▫▫▫▩*" fail
Shrink #3: "▫Ѧ▫▫▫▩" fail
Shrink #4: "▫Ѧ▩" fail
Shrink #5: "▫Ѧ" fail
Shrink #6: "▩" fail
Shrink #7: <empty string> pass
Shrink result (after 7 shrinks) => "▩"
```

And there is more!

Error!

```
Attempting to shrink arg "▫ℳ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫Ấ▫▫▫▫▫Ở▫▫▫▫▫▫▫▫▫ḧ▩▩▩▩"
Shrink #1: "▫ℳ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▩▩▩" fail
Shrink #2: "▫ℳ▫▫▫▫▫▫▫▫▩∗" fail
Shrink #3: "▫ℳ▫▫▫▩" fail
Shrink #4: "▫ℳ▩" fail
Shrink #5: "▫ℳ" fail
Shrink #6: "▩" fail
Shrink #7: <empty string> pass
Shrink result (after 7 shrinks) => "▩"
```

And there is more!

Error!

```
Attempting to shrink arg "▫ℳ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫Ấ▫▫▫▫▫Ở▫▫▫▫▫▫▫▫▫ỵ⊠⊠⊠⊠"
Shrink #1: "▫ℳ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫⊠⊠⊠" fail
Shrink #2: "▫ℳ▫▫▫▫▫▫▫▫⊠∗" fail
Shrink #3: "▫ℳ▫▫▫⊠" fail
Shrink #4: "▫ℳ⊠" fail
Shrink #5: "▫ℳ" fail
Shrink #6: "⊠" fail
Shrink #7: <empty string> pass
Shrink result (after 7 shrinks) => "⊠"
```

And there is more!

Error!

```
Attempting to shrink arg "▫Ʌ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫Ấ▫▫▫▫▫Ở▫▫▫▫▫▫▫▫▫ṉ▩▩▩▩"
Shrink #1: "▫Ʌ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▩▩▩" fail
Shrink #2: "▫Ʌ▫▫▫▫▫▫▫▫▩*" fail
Shrink #3: "▫Ʌ▫▫▫▩" fail
Shrink #4: "▫Ʌ▩" fail
Shrink #5: "▫Ʌ" fail
Shrink #6: "▩" fail
Shrink #7: <empty string> pass
Shrink result (after 7 shrinks) => "▩"
```

And there is more!

Error!

```
Attempting to shrink arg "▫Ѧ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫Ấ▫▫▫▫▫Ở▫▫▫▫▫▫▫▫▫ḧ▩▩▩▩"
Shrink #1: "▫Ѧ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▩▩▩" fail
Shrink #2: "▫Ѧ▫▫▫▫▫▫▫▫▩*" fail
Shrink #3: "▫Ѧ▫▫▫▩" fail
Shrink #4: "▫Ѧ▩" fail
Shrink #5: "▫Ѧ" fail
Shrink #6: "▩" fail
Shrink #7: <empty string> pass
Shrink result (after 7 shrinks) => "▩"
```

And there is more!

Error!

```
Property failed after 1 attempts

  Arg 0: "⊠" (shrunk from ▫Ⲗ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫Ẩ▫▫▫▫▫Ở▫▫▫▫▫▫▫▫▫ӈ▫▫▫▫)

Repeat this test by using seed 4919313423746630853
```

Error!

```
Property failed after 1 attempts

  Arg 0: "⊠" (shrunk from ▫Ñ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫Ấ▫▫▫▫▫Ở▫▫▫▫▫▫▫▫▫Ņ▫▫▫▫)

Repeat this test by using seed 4919313423746630853
```

Error!

```
Property failed after 1 attempts

  Arg 0: "⊠" (shrunk from ▫Ⲗ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫Ấ▫▫▫▫▫Ở▫▫▫▫▫▫▫▫▫ӈ▫▫▫▫)

Repeat this test by using seed 4919313423746630853
```

Error!

```
Property failed after 1 attempts

  Arg 0: "⊠" (shrunk from ▫Ñ▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫Ấ▫▫▫▫▫Ở▫▫▫▫▫▫▫▫▫Ậ▫▫▫▫)

Repeat this test by using seed 4919313423746630853
```

Repeat? Yes!

```
checkAll(
    PropTestConfig(seed = 4919313423746630853),
    Arb.string(
        codepoints = Arb.of(
            ('a'.code..Char.MAX_HIGH_SURROGATE.code)
                .map(::Codepoint)
        ),
    )
) { input ->
    MD5.compute(input) shouldBe libMd5(input)
}
```

- reproduce the test **exactly** like in the failed instance
- debug it
- fix it
- PROFIT!!

Repeat? Yes!

```
checkAll(
    PropTestConfig(seed = 4919313423746630853),
    Arb.string(
        codepoints = Arb.of(
            ('a'.code..Char.MAX_HIGH_SURROGATE.code)
                .map(::Codepoint)
        ),
    )
) { input ->
    MD5.compute(input) shouldBe libMd5(input)
}
```

- reproduce the test **exactly** like in the failed instance
- debug it
- fix it
- PROFIT!!




Time to mock!

Disclaimer

I believe one should mock, not fake. Because

1. Granular control
2. Can check that something was NOT called

Time to mock

Setup

```
class UserService(val repo: UserRepository) {
    fun getAllUsers() = repo.allUsers()
}

class UserRepository {
    fun allUsers(): List<User> { TODO("demo") }
}

class User(val name: String, val age: Int)
```

Time to mock

Setup

```
class UserService(val repo: UserRepository) {
    fun getAllUsers() = repo.allUsers()
}

class UserRepository {
    fun allUsers(): List<User> { TODO("demo") }
}

class User(val name: String, val age: Int)
```

Time to mock

Setup

```
class UserService(val repo: UserRepository) {
    fun getAllUsers() = repo.allUsers()
}

class UserRepository {
    fun allUsers(): List<User> { TODO("demo") }
}

class User(val name: String, val age: Int)
```

Time to mock

Setup

```
class UserService(val repo: UserRepository) {
    fun getAllUsers() = repo.allUsers()
}

class UserRepository {
    fun allUsers(): List<User> { TODO("demo") }
}

class User(val name: String, val age: Int)
```

Time to mock

Setup

```
class UserService(val repo: UserRepository) {
    fun getAllUsers() = repo.allUsers()
}

class UserRepository {
    fun allUsers(): List<User> { TODO("demo") }
}

class User(val name: String, val age: Int)
```

Time to mock

Setup

```
class UserService(val repo: UserRepository) {
    fun getAllUsers() = repo.allUsers()
}

class UserRepository {
    fun allUsers(): List<User> { TODO("demo") }
}

class User(val name: String, val age: Int)
```

Time to mock

Setup

```
class UserService(val repo: UserRepository) {
    fun getAllUsers() = repo.allUsers()
}

class UserRepository {
    fun allUsers(): List<User> { TODO("demo") }
}

class User(val name: String, val age: Int)
```

Time to mock

Setup

```
class UserService(val repo: UserRepository) {
    fun getAllUsers() = repo.allUsers()
}

class UserRepository {
    fun allUsers(): List<User> { TODO("demo") }
}

class User(val name: String, val age: Int)
```

How the test looks in Java

```java
UserRepository mockRepository = Mockito.mock(UserRepository.class);

// Create some sample user data
List<User> users = Arrays.asList(new User("Alice", 25), new User("Bob", 30), new User("Charlie", 35));

// Define the expected behavior of the mock repository
Mockito.when(mockRepository.allUsers()).thenReturn(users);

// Create an instance of the UserService with the mock repository
UserService userService = new UserService(mockRepository);

// Call the method under test
List<User> result = userService.getAllUsers();

// Verify the result
Assertions.assertEquals(users, result);
```

How the test looks in Java

```java
UserRepository mockRepository = Mockito.mock(UserRepository.class);

// Create some sample user data
List<User> users = Arrays.asList(new User("Alice", 25), new User("Bob", 30), new User("Charlie", 35));

// Define the expected behavior of the mock repository
Mockito.when(mockRepository.allUsers()).thenReturn(users);

// Create an instance of the UserService with the mock repository
UserService userService = new UserService(mockRepository);

// Call the method under test
List<User> result = userService.getAllUsers();

// Verify the result
Assertions.assertEquals(users, result);
```

How the test looks in Java

```java
UserRepository mockRepository = Mockito.mock(UserRepository.class);

// Create some sample user data
List<User> users = Arrays.asList(new User("Alice", 25), new User("Bob", 30), new User("Charlie", 35));

// Define the expected behavior of the mock repository
Mockito.when(mockRepository.allUsers()).thenReturn(users);

// Create an instance of the UserService with the mock repository
UserService userService = new UserService(mockRepository);

// Call the method under test
List<User> result = userService.getAllUsers();

// Verify the result
Assertions.assertEquals(users, result);
```

How the test looks in Java

```
UserRepository mockRepository = Mockito.mock(UserRepository.class);

// Create some sample user data
List<User> users = Arrays.asList(new User("Alice", 25), new User("Bob", 30), new User("Charlie", 35));

// Define the expected behavior of the mock repository
Mockito.when(mockRepository.allUsers()).thenReturn(users);

// Create an instance of the UserService with the mock repository
UserService userService = new UserService(mockRepository);

// Call the method under test
List<User> result = userService.getAllUsers();

// Verify the result
Assertions.assertEquals(users, result);
```

How the test looks in Java

```
UserRepository mockRepository = Mockito.mock(UserRepository.class);

// Create some sample user data
List<User> users = Arrays.asList(new User("Alice", 25), new User("Bob", 30), new User("Charlie", 35));

// Define the expected behavior of the mock repository
Mockito.when(mockRepository.allUsers()).thenReturn(users);

// Create an instance of the UserService with the mock repository
UserService userService = new UserService(mockRepository);

// Call the method under test
List<User> result = userService.getAllUsers();

// Verify the result
Assertions.assertEquals(users, result);
```

How the test looks in Java

```java
UserRepository mockRepository = Mockito.mock(UserRepository.class);

// Create some sample user data
List<User> users = Arrays.asList(new User("Alice", 25), new User("Bob", 30), new User("Charlie", 35));

// Define the expected behavior of the mock repository
Mockito.when(mockRepository.allUsers()).thenReturn(users);

// Create an instance of the UserService with the mock repository
UserService userService = new UserService(mockRepository);

// Call the method under test
List<User> result = userService.getAllUsers();

// Verify the result
Assertions.assertEquals(users, result);
```

How the test looks in Java

```java
UserRepository mockRepository = Mockito.mock(UserRepository.class);

// Create some sample user data
List<User> users = Arrays.asList(new User("Alice", 25), new User("Bob", 30), new User("Charlie", 35));

// Define the expected behavior of the mock repository
Mockito.when(mockRepository.allUsers()).thenReturn(users);

// Create an instance of the UserService with the mock repository
UserService userService = new UserService(mockRepository);

// Call the method under test
List<User> result = userService.getAllUsers();

// Verify the result
Assertions.assertEquals(users, result);
```

Time to mock! (MockK edition)

```
val users = listOf(new User("Alice", 25), new User("Bob", 30), new User("Charlie", 35))
val repo = mockk<UserRepository> {
    every { allUsers() } returns users
}
val service = UserService(repo)
service.getAllUsers() shouldBe users
```

Time to mock! (MockK edition)

```
val users = listOf(new User("Alice", 25), new User("Bob", 30), new User("Charlie", 35))
val repo = mockk<UserRepository> {
    every { allUsers() } returns users
}
val service = UserService(repo)
service.getAllUsers() shouldBe users
```

Time to mock! (MockK edition)

```
val users = listOf(new User("Alice", 25), new User("Bob", 30), new User("Charlie", 35))
val repo = mockk<UserRepository> {
    every { allUsers() } returns users
}
val service = UserService(repo)
service.getAllUsers() shouldBe users
```

Time to mock! (MockK edition)

```
val users = listOf(new User("Alice", 25), new User("Bob", 30), new User("Charlie", 35))
val repo = mockk<UserRepository> {
    every { allUsers() } returns users
}
val service = UserService(repo)
service.getAllUsers() shouldBe users
```

Time to mock! (MockK edition)

```
val users = listOf(new User("Alice", 25), new User("Bob", 30), new User("Charlie", 35))
val repo = mockk<UserRepository> {
    every { allUsers() } returns users
}
val service = UserService(repo)
service.getAllUsers() shouldBe users
```

Time to mock! (MockK edition)

```
val users = listOf(new User("Alice", 25), new User("Bob", 30), new User("Charlie", 35))
val repo = mockk<UserRepository> {
    every { allUsers() } returns users
}
val service = UserService(repo)
service.getAllUsers() shouldBe users
```

Time to mock! (MockK edition)

```
1   val users = listOf(new User("Alice", 25), new User("Bob", 30), new User("Charlie", 35))
2   val repo = mockk<UserRepository> {
3       every { allUsers() } returns users
4   }
5   val service = UserService(repo)
6   service.getAllUsers() shouldBe users
```

Nice DSL

Code describing `` `UserRepository` `` is inside `` `{}` ``

What if I want to mock users too?

Time to mock users! (mockk edition)

```
val repo = mockk<UserRepository> {
    every { allUsers() } returns listOf(
        mockk {
            every { name } returns "Pasha"
        },
        mockk {
            every { name } returns "Alexandra"
        },
    )
}
val service = UserService(repo)
service.getAllUsers()[1].name shouldBe "Alexandra"
```

Thank Alexandra for helping me with this presentation!

Time to mock users! (mockk edition)

```
val repo = mockk<UserRepository> {
    every { allUsers() } returns listOf(
        mockk {
            every { name } returns "Pasha"
        },
        mockk {
            every { name } returns "Alexandra"
        },
    )
}
val service = UserService(repo)
service.getAllUsers()[1].name shouldBe "Alexandra"
```

Thank Alexandra for helping me with this presentation!

Time to mock users! (mockk edition)

```
val repo = mockk<UserRepository> {
    every { allUsers() } returns listOf(
        mockk {
            every { name } returns "Pasha"
        },
        mockk {
            every { name } returns "Alexandra"
        },
    )
}
val service = UserService(repo)
service.getAllUsers()[1].name shouldBe "Alexandra"
```

Thank Alexandra for helping me with this presentation!

Time to mock users! (mockk edition)

```
val repo = mockk<UserRepository> {
    every { allUsers() } returns listOf(
        mockk {
            every { name } returns "Pasha"
        },
        mockk {
            every { name } returns "Alexandra"
        },
    )
}
val service = UserService(repo)
service.getAllUsers()[1].name shouldBe "Alexandra"
```

Thank Alexandra for helping me with this presentation!

Time to mock users! (mockk edition)

```
val repo = mockk<UserRepository> {
    every { allUsers() } returns listOf(
        mockk {
            every { name } returns "Pasha"
        },
        mockk {
            every { name } returns "Alexandra"
        },
    )
}
val service = UserService(repo)
service.getAllUsers()[1].name shouldBe "Alexandra"
```

Thank Alexandra for helping me with this presentation!

Time to mock users! (mockk edition)

```
val repo = mockk<UserRepository> {
    every { allUsers() } returns listOf(
        mockk {
            every { name } returns "Pasha"
        },
        mockk {
            every { name } returns "Alexandra"
        },
    )
}
val service = UserService(repo)
service.getAllUsers()[1].name shouldBe "Alexandra"
```

Thank Alexandra for helping me with this presentation!

Time to mock users! (mockk edition)

```
val repo = mockk<UserRepository> {
    every { allUsers() } returns listOf(
        mockk {
            every { name } returns "Pasha"
        },
        mockk {
            every { name } returns "Alexandra"
        },
    )
}
val service = UserService(repo)
service.getAllUsers()[1].name shouldBe "Alexandra"
```

Thank Alexandra for helping me with this presentation!

Users are described inside `` `User` `` mocks!

Atrium expectations!

https://docs.atriumlib.org/

```
expect(service.getAllUsers()) toContain o inAny order but only the entries(
    {
        its { name } toEqual "Pash"
        its { age } toEqual 36
    },
    {
        its { name } toEqual "Mark"
        its { age } toEqual 13
    }
)
```

Atrium expectations!

https://docs.atriumlib.org/

```
expect(service.getAllUsers()) toContain o inAny order but only the entries(
    {
        its { name } toEqual "Pash"
        its { age } toEqual 36
    },
    {
        its { name } toEqual "Mark"
        its { age } toEqual 13
    }
)
```

Atrium expectations!

https://docs.atriumlib.org/

```
expect(service.getAllUsers()) toContain o inAny order but only the entries(
    {
        its { name } toEqual "Pash"
        its { age } toEqual 36
    },
    {
        its { name } toEqual "Mark"
        its { age } toEqual 13
    }
)
```

I’ve made a mistake to show the error message

Atrium expectations!

https://docs.atriumlib.org/

```
expect(service.getAllUsers()) toContain o inAny order but only the entries(
    {
        its { name } toEqual "Pash"
        its { age } toEqual 36
    },
    {
        its { name } toEqual "Mark"
        its { age } toEqual 13
    }
)
```

I’ve made a mistake to show the error message

Atrium expectations!

https://docs.atriumlib.org/

```
expect(service.getAllUsers()) toContain o inAny order but only the entries(
    {
        its { name } toEqual "Pash"
        its { age } toEqual 36
    },
    {
        its { name } toEqual "Mark"
        its { age } toEqual 13
    }
)
```

I’ve made a mistake to show the error message

Error report

```
I expected subject: [User(#3), User(#4)]        (java.util.Arrays.ArrayList <315506631>)
◆ to contain only, in any order:
  ✘ an element which needs:
      » ▶ its.definedIn(MockTest.kt:42):
          ■ to equal: "Pash"        <2026086646>
      » ▶ its.definedIn(MockTest.kt:43):
          ■ to equal: 36        (kotlin.Int <116289638>)
  ✔ an element which needs:
      » ▶ its.definedIn(MockTest.kt:46):
          ■ to equal: "Mark"        <574921197>
      » ▶ its.definedIn(MockTest.kt:47):
          ■ to equal: 13        (kotlin.Int <453296736>)
    ❗❗ following elements were mismatched:
      ∘ User(#3)        (User <1041341319>)
```

Error report

```
I expected subject: [User(#3), User(#4)]        (java.util.Arrays.ArrayList <315506631>)
◆ to contain only, in any order:
  ✘ an element which needs:
      » ▶ its.definedIn(MockTest.kt:42):
          ■ to equal: "Pash"        <2026086646>
      » ▶ its.definedIn(MockTest.kt:43):
          ■ to equal: 36        (kotlin.Int <116289638>)
  ✔ an element which needs:
      » ▶ its.definedIn(MockTest.kt:46):
          ■ to equal: "Mark"        <574921197>
      » ▶ its.definedIn(MockTest.kt:47):
          ■ to equal: 13        (kotlin.Int <453296736>)
    ❗❗ following elements were mismatched:
      ◦ User(#3)        (User <1041341319>)
```

Error report

```
I expected subject: [User(#3), User(#4)]        (java.util.Arrays.ArrayList <315506631>)
◆ to contain only, in any order:
  ✘ an element which needs:
      » ▶ its.definedIn(MockTest.kt:42):
          ■ to equal: "Pash"        <2026086646>
      » ▶ its.definedIn(MockTest.kt:43):
          ■ to equal: 36        (kotlin.Int <116289638>)
  ✔ an element which needs:
      » ▶ its.definedIn(MockTest.kt:46):
          ■ to equal: "Mark"        <574921197>
      » ▶ its.definedIn(MockTest.kt:47):
          ■ to equal: 13        (kotlin.Int <453296736>)
    ❗❗ following elements were mismatched:
      ◦ User(#3)        (User <1041341319>)
```

Error report

```
I expected subject: [User(#3), User(#4)]        (java.util.Arrays.ArrayList <315506631>)
◆ to contain only, in any order:
  ✘ an element which needs:
      » ▶ its.definedIn(MockTest.kt:42):
          ■ to equal: "Pash"        <2026086646>
      » ▶ its.definedIn(MockTest.kt:43):
          ■ to equal: 36        (kotlin.Int <116289638>)
  ✔ an element which needs:
      » ▶ its.definedIn(MockTest.kt:46):
          ■ to equal: "Mark"        <574921197>
      » ▶ its.definedIn(MockTest.kt:47):
          ■ to equal: 13        (kotlin.Int <453296736>)
    ! ! following elements were mismatched:
      ∘ User(#3)        (User <1041341319>)
```

Error report

```
I expected subject: [User(#3), User(#4)]        (java.util.Arrays.ArrayList <315506631>)
◆ to contain only, in any order:
  ✘ an element which needs:
      » ▶ its.definedIn(MockTest.kt:42):
          ■ to equal: "Pash"        <2026086646>
      » ▶ its.definedIn(MockTest.kt:43):
          ■ to equal: 36        (kotlin.Int <116289638>)
  ✔ an element which needs:
      » ▶ its.definedIn(MockTest.kt:46):
          ■ to equal: "Mark"        <574921197>
      » ▶ its.definedIn(MockTest.kt:47):
          ■ to equal: 13        (kotlin.Int <453296736>)
    ❗❗ following elements were mismatched:
      ◦ User(#3)        (User <1041341319>)
```

Error report

```
I expected subject: [User(#3), User(#4)]        (java.util.Arrays.ArrayList <315506631>)
◆ to contain only, in any order:
  ✘ an element which needs:
      » ▶ its.definedIn(MockTest.kt:42):
          ■ to equal: "Pash"        <2026086646>
      » ▶ its.definedIn(MockTest.kt:43):
          ■ to equal: 36        (kotlin.Int <116289638>)
  ✔ an element which needs:
      » ▶ its.definedIn(MockTest.kt:46):
          ■ to equal: "Mark"        <574921197>
      » ▶ its.definedIn(MockTest.kt:47):
          ■ to equal: 13        (kotlin.Int <453296736>)
    ❗❗ following elements were mismatched:
      ∘ User(#3)        (User <1041341319>)
```

Error report

```
I expected subject: [User(#3), User(#4)]        (java.util.Arrays.ArrayList <315506631>)
◆ to contain only, in any order:
  ✘ an element which needs:
      » ▶ its.definedIn(MockTest.kt:42):
          ■ to equal: "Pash"        <2026086646>
      » ▶ its.definedIn(MockTest.kt:43):
          ■ to equal: 36        (kotlin.Int <116289638>)
  ✔ an element which needs:
      » ▶ its.definedIn(MockTest.kt:46):
          ■ to equal: "Mark"        <574921197>
      » ▶ its.definedIn(MockTest.kt:47):
          ■ to equal: 13        (kotlin.Int <453296736>)
    ❗❗ following elements were mismatched:
      ◦ User(#3)        (User <1041341319>)
```

- Readable as a text
- Shows the errors

Dessert

Now, when we saw different nice DSL-using capabilities…

```
ideaTest<App>("Create Kafka connection") {
    showBDTPanel {
        createBaseConnection("Kafka") {
            fillCredentialsForKafka(connectionName)
            checkSuccessfulMessage()
            ok()
        }
        checkMonitoringConnection(monitoringToolWindow)
    }
}
```

How does it work? Magick!

Dive deeper



What did we learn?



What did we learn?

1. Tests should be readable
2. Testing is not "just unit tests" sometimes
3. Kotest gives good structure to the tests.
4. Kotest has built-in property tests
5. Atrium gives us assertions DSL

Thank you! Time for questions

😄



asm0di0

@asm0dey@fosstodon.org

me@asm0dey.site

https://jb.gg/twitch

asm0dey

https://linktr.ee/asm0dey



END

Deeper dive into JetBrains tests

```
ideaTest<App>("Create Kafka connection") {
```

really means:

```
fun <T : App> Group.ideaTest(title: String,
                             enabled: Boolean = true,
                             isMuted: Boolean = false,
                             testBody: T.(ctx: IdeaNodeContext<T>) -> Unit) {
    IdeaNodeContext<T>(scenarioContext).apply {
        test(title, enabled, isMuted) {
            app.testBody(this)
        }
    }
}
```

Deeper dive into JetBrains tests

```
ideaTest<App>("Create Kafka connection") {
```

really means:

```
fun <T : App> Group.ideaTest(title: String,
                             enabled: Boolean = true,
                             isMuted: Boolean = false,
                             testBody: T.(ctx: IdeaNodeContext<T>) -> Unit) {
    IdeaNodeContext<T>(scenarioContext).apply {
        test(title, enabled, isMuted) {
            app.testBody(this)
        }
    }
}
```

Deeper dive into JetBrains tests

```
ideaTest<App>("Create Kafka connection") {
```

really means:

```
fun <T : App> Group.ideaTest(title: String,
                             enabled: Boolean = true,
                             isMuted: Boolean = false,
                             testBody: T.(ctx: IdeaNodeContext<T>) -> Unit) {
    IdeaNodeContext<T>(scenarioContext).apply {
        test(title, enabled, isMuted) {
            app.testBody(this)
        }
    }
}
```

Deeper dive into JetBrains tests

```
ideaTest<App>("Create Kafka connection") {
```

really means:

```
fun <T : App> Group.ideaTest(title: String,
                             enabled: Boolean = true,
                             isMuted: Boolean = false,
                             testBody: T.(ctx: IdeaNodeContext<T>) -> Unit) {
    IdeaNodeContext<T>(scenarioContext).apply {
        test(title, enabled, isMuted) {
            app.testBody(this)
        }
    }
}
```

Deeper dive into JetBrains tests

```
1       showBDTPanel {
```

really means:

```
1   fun App.showBDTPanel(block: BDTPanel.() -> Unit) {
2       val app = this
3       ideaFrame {
4           val frame = this
5           step("Open Big Data Tools Panel") {
6               openBDTPanel(app)
7               if (isKafka()) {
8                   try {
9                       container(byXpath("//div[@accessiblename='Kafka Tool Window' and @class='InternalDecoratorImpl']"
10                          BDTPanel(app, frame, this).block()
11                      }
```

Deeper dive into JetBrains tests

```
1       showBDTPanel {
```

really means:

```
1   fun App.showBDTPanel(block: BDTPanel.() -> Unit) {
2       val app = this
3       ideaFrame {
4           val frame = this
5           step("Open Big Data Tools Panel") {
6               openBDTPanel(app)
7               if (isKafka()) {
8                   try {
9                       container(byXpath("//div[@accessiblename='Kafka Tool Window' and @class='InternalDecoratorImpl']"
10                          BDTPanel(app, frame, this).block()
11                      }
```

Deeper dive into JetBrains tests

```
showBDTPanel {
```

really means:

```
fun App.showBDTPanel(block: BDTPanel.() -> Unit) {
    val app = this
    ideaFrame {
        val frame = this
        step("Open Big Data Tools Panel") {
            openBDTPanel(app)
            if (isKafka()) {
                try {
                    container(byXpath("//div[@accessiblename='Kafka Tool Window' and @class='InternalDecoratorImpl']"
                        BDTPanel(app, frame, this).block()
                    }
```

Deeper dive into JetBrains tests

```
1       showBDTPanel {
```

really means:

```
1   fun App.showBDTPanel(block: BDTPanel.() -> Unit) {
2       val app = this
3       ideaFrame {
4           val frame = this
5           step("Open Big Data Tools Panel") {
6               openBDTPanel(app)
7               if (isKafka()) {
8                   try {
9                       container(byXpath("//div[@accessiblename='Kafka Tool Window' and @class='InternalDecoratorImpl']"
10                          BDTPanel(app, frame, this).block()
11                      }
```

Deeper dive into JetBrains tests

```
1       showBDTPanel {
```

really means:

```
1   fun App.showBDTPanel(block: BDTPanel.() -> Unit) {
2       val app = this
3       ideaFrame {
4           val frame = this
5           step("Open Big Data Tools Panel") {
6               openBDTPanel(app)
7               if (isKafka()) {
8                   try {
9                       container(byXpath("//div[@accessiblename='Kafka Tool Window' and @class='InternalDecoratorImpl']"
10                          BDTPanel(app, frame, this).block()
11                      }
```

Deeper dive into JetBrains tests

```
1       showBDTPanel {
```

really means:

```
1   fun App.showBDTPanel(block: BDTPanel.() -> Unit) {
2       val app = this
3       ideaFrame {
4           val frame = this
5           step("Open Big Data Tools Panel") {
6               openBDTPanel(app)
7               if (isKafka()) {
8                   try {
9                       container(byXpath("//div[@accessiblename='Kafka Tool Window' and @class='InternalDecoratorImpl']"
10                          BDTPanel(app, frame, this).block()
11                      }
```

Deeper dive into JetBrains tests

```
1           createBaseConnection("Kafka") {
```

really means:

```
1   fun createBaseConnection(typeOfConnection: String, block: ConnectionDialog.() -> Unit) {
2       step("Open connection dialog") {
3           if (app.isKafka()) {
4               frame.find<ComponentFixture>(byXpath("//div[contains(@tooltiptext.key, 'toolwindow.tab.add')]")).click()
5               clickTypeOfConnection(typeOfConnection, block)
6               pause(5000)
```

Deeper dive into JetBrains tests

```
1           createBaseConnection("Kafka") {
```

really means:

```
1   fun createBaseConnection(typeOfConnection: String, block: ConnectionDialog.() -> Unit) {
2       step("Open connection dialog") {
3           if (app.isKafka()) {
4               frame.find<ComponentFixture>(byXpath("//div[contains(@tooltiptext.key, 'toolwindow.tab.add')]")).click()
5               clickTypeOfConnection(typeOfConnection, block)
6               pause(5000)
```

Deeper dive into JetBrains tests

```
1           createBaseConnection("Kafka") {
```

really means:

```
1   fun createBaseConnection(typeOfConnection: String, block: ConnectionDialog.() -> Unit) {
2       step("Open connection dialog") {
3           if (app.isKafka()) {
4               frame.find<ComponentFixture>(byXpath("//div[contains(@tooltiptext.key, 'toolwindow.tab.add')]")).click()
5               clickTypeOfConnection(typeOfConnection, block)
6               pause(5000)
```

Deeper dive into JetBrains tests

```
1           createBaseConnection("Kafka") {
```

really means:

```
1   fun createBaseConnection(typeOfConnection: String, block: ConnectionDialog.() -> Unit) {
2       step("Open connection dialog") {
3           if (app.isKafka()) {
4               frame.find<ComponentFixture>(byXpath("//div[contains(@tooltiptext.key, 'toolwindow.tab.add')]")).click()
5               clickTypeOfConnection(typeOfConnection, block)
6               pause(5000)
```

Deeper dive into JetBrains tests

```
1           createBaseConnection("Kafka") {
```

really means:

```
1   fun createBaseConnection(typeOfConnection: String, block: ConnectionDialog.() -> Unit) {
2       step("Open connection dialog") {
3           if (app.isKafka()) {
4               frame.find<ComponentFixture>(byXpath("//div[contains(@tooltiptext.key, 'toolwindow.tab.add')]")).click()
5               clickTypeOfConnection(typeOfConnection, block)
6               pause(5000)
```

And this is the abyss!

But you can do the same!

Go back to summary

END